CODERHOUSE / DATA ENGINEERING

Redshift
Streaming Analytics

Nicolás López · Pre-entrega 6

Corrección y ejecución completa desde una base limpia

Verificación real: 11 de noviembre de 2026

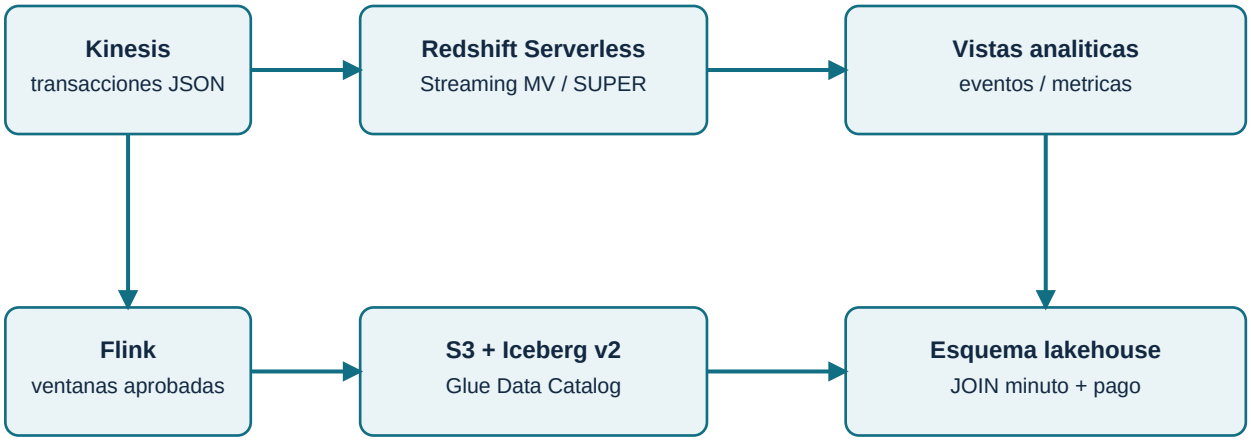
SQL01-08: 42 sentencias finalizadas. Nueve casos de conversion verificados. Ingesta, metricas, JOIN y permisos comprobados.

Prueba y retiro verificados; capturas actuales incorporadas.

Repositorio

Código de esta revisión

1. Arquitectura y alcance



Ramas existentes: Kinesis / Flink / Lakehouse. Redshift: ensayo temporal validado en AWS.

Kinesis alimenta directamente la MV de Redshift. Flink es un consumidor independiente y produce agregaciones aprobadas Iceberg en S3, registradas en Glue. Redshift consulta ambas rutas mediante SQL y un esquema externo de Data Catalog.

En esta revision se recreo solamente el modulo temporal de Redshift: namespace, workgroup privado de 4 RPU, rol/politica IAM, grupos de seguridad, endpoint Kinesis y limite de uso. Terraform: ocho altas, cero cambios y cero bajas. El Lakehouse y Kinesis existentes se conservaron.

Flink permanecio READY, sin iniciar. Se publicaron llegadas nuevas con Event Time actual e historico para contrastar metricas aprobadas equivalentes contra el snapshot PE5. Esto verifica ingesta y consulta conjunta; no es una prueba simultanea nueva de escritura Flink.

2. Correcciones y reproducibilidad

Punto revisado	Resultado
Campos y CTE	payment_method correcto; CTE con parentesis. Vistas creadas realmente en Redshift.
Validacion monetaria	NOT (amount_text ~ patron), NULL explicito y TRY_CAST.
Timestamp	Epoch ms o ISO con zona; conversion UTC y rechazo de fechas invalidas.
Monitoreo	SYS_STREAM_SCAN_ERRORS y demas consultas ejecutadas.
Membresia	SQL08 asigna rol al usuario sin password y prueba identidad efectiva.
Version exacta	Fuente consolidada, hash y recibos por sentencia enlazados.

3. Ejecución desde cero

La precondicion consulto pg_namespace y comprobo ausencia de streaming, analytics, kinesis_source y lakehouse. No se borraron esquemas para simular una base limpia. Se ejecutaron SQL01-05, se publico el fixture y se refresco la MV; despues se ejecutaron todas las sentencias SQL06-08 en una sesion retenida.

Fuente	Sentencias FINISHED
01_schemas.sql	5
02_streaming.sql	1
03_analytics.sql	2
04_iceberg.sql	1
05_security.sql	10
06_validation.sql	7
07_monitoring.sql	7
08_membership.sql	9

Los recibos estan en docs/evidence/checkpoint-6/revision. En los dos CREATE EXTERNAL SCHEMA Redshift oculta IAM_ROLE y REGION en el texto devuelto: la comparacion de esos recibos se limita al prefijo visible. Las restantes sentencias coinciden con el SQL devuelto; no se afirma comparacion byte a byte de campos ocultos.

SHA256 del consolidado: b51176a029026b80b8bbac5f6147fbe56ab724ec7b1df64e811b174c0ec8e30c

RunId: pe6_20261111_222257. No se republico el lote para obtener capturas.

4. Parseo JSON y conversiones

La MV conserva bytes/raw, coordenadas del stream y payload SUPER. CAN_JSON_PARSE guarda el JSON invalido sin ejecutar JSON_PARSE sobre el. La capa transactions_typed extrae cinco campos con tipos adecuados; la validacion ocurre antes de agregar.

Campo JSON	Salida	Tipo real observado
transaction_id	transaction_id	text
timestamp	event_time	timestamp without time zone
amount	amount	numeric
payment_method	payment_method	text
status	status	text

amount se define decimal(18,2). El AVG amplia el argumento a decimal(24,6) antes de agregar y publica decimal(38,6); un cast final por si solo no recupera decimales perdidos.

5. Regresión de timestamp e importe

Se ejecuto el SELECT real de SQL03 sobre una tabla TEMP en Redshift, en una sesion independiente. No es una simulacion Python ni solo un chequeo de gramatica.

Caso	Event Time UTC observado	Clasificacion
bad-date	NULL	invalid_timestamp
bad-money	2026-11-11 15:00:00	invalid_amount
empty-time	NULL	invalid_timestamp
epoch-ms	2026-11-11 15:00:00	valido
iso-offset	2026-11-11 15:00:00	valido
iso-z	2026-11-11 15:00:00	valido
missing-money	2026-11-11 15:00:00	invalid_amount
missing-time	NULL	invalid_timestamp
no-timezone	NULL	invalid_timestamp

ISO Z, offset -03:00 y epoch ms convergen a 2026-11-11 15:00:00 UTC. La fecha imposible, ausencia de zona, vacio y faltante no producen errores de casting; quedan clasificados. Se requieren zona explicita y hasta seis decimales de segundo.

6. Ingesta y métricas observadas

Metrica	Esperado	Observado
Registros Kinesis del lote	9	9
Validos tipados	6	6
Invalidos clasificados	3	3
Transacciones del grupo actual	3	3
Aprobadas / rechazadas	2 / 1	2 / 1
Importe total	1029.30	1029.30
Importe aprobado	30.30	30.30
Promedio	343.100000	343.100000

La muestra incluye dos aprobadas (10.10 y 20.20), una rechazada (999), importe invalido, timestamp faltante y JSON invalido. Se preservan recibos PutRecord y shard/sequence_number. arrival_timestamp procede de Kinesis, no del reloj del productor.

7. JOIN con Iceberg

Metrica equivalente	Stream	Iceberg
Ventana UTC	2026-11-04 11:33:00	2026-11-04 11:33:00
Aprobadas	2	2
Importe aprobado	30.30	30.30
Diferencia cantidad	0	0
Diferencia importe	0.00	0.00

El JOIN usa minuto UTC y metodo de pago. Iceberg contiene agregaciones aprobadas, no eventos raw. Se excluyen rechazadas de la comparacion. La llegada del ensayo es nueva aunque el Event Time historico sea del 04/11/2026. No se suman dos historias solapadas ni se promete exactly-once entre consumidores independientes.

Glue GetTable y el JSON real de metadatos S3 se volvieron a leer en esta sesion. Formato observado: ICEBERG. Snapshot existente: 8171898520026477263. El snapshot PE5 se conservo; no se fabrico una tabla Glue ni se escribieron datos nuevos con Flink.

8. IAM, rol SQL y membresía efectiva

El rol de ejecucion Redshift usa IAM para Kinesis, KMS, Glue y S3, con ARN/prefijo acotados. Incluye DescribeStream, GetShardIterator y GetRecords. La red es privada; el endpoint Kinesis pertenece al modulo temporal. El rol SQL checkpoint6_analyst separa la lectura analitica de permisos de propietario y refresco.

Usuario observado	Rol	admin_option
checkpoint6_access_test	checkpoint6_analyst	False
IAM:RootIdentity	checkpoint6_analyst	False

Identidad efectiva probada: checkpoint6_access_test. SELECT sobre metricas y JOIN permitido; acceso raw y REFRESH denegados. Las dos respuestas FAILED de esas pruebas son denegaciones esperadas y se conservaron. No son errores de sintaxis del script consolidado.

9. Refresco, monitoreo y operación

Ensayo: AUTO REFRESH NO, publicacion unica, REFRESH explicito y consultas despues de su finalizacion. El refresco incremental se verifica con SYS_MV_REFRESH_HISTORY; SYS_STREAM_SCAN_STATES/ERRORS muestran escaneos, posiciones y errores. El intervalo observado incluye llamadas API y diagnosticos, no es una garantia de latencia automatica.

Desde el inicio del envio del lote hasta completar sus diagnosticos se observaron 95.28 segundos. No equivale a la duracion exclusiva del refresco ni a un SLA de ingesta.

Produccion propuesta: orquestador cada 60 segundos, sin solapar refrescos. Conservar statement_id; ante timeout consultar su estado antes de repetir. Reintentar solo throttling/indisponibilidad hasta tres veces, con esperas 5/15/30 segundos y jitter. Detener y alertar ante sintaxis, permisos o casting persistentes. El orquestador es una politica documentada; no se desplego en esta prueba.

Habilitar AUTO REFRESH YES solo tras observar frescura y costo en el workgroup real. Deshabilitarlo al activar el orquestador, terminar el ensayo o detectar errores/gasto persistentes. AUTO REFRESH no implica un SLA de 60 segundos.

10. Retiro y limitaciones

Retiro temporal: confirmado. El plan de limpieza permite bajas exclusivamente dentro de module.redshift[0]. No se elimina el stream, Flink, Glue, Iceberg, VPC ni S3 de entregas previas. Su almacenamiento/stream/logs pueden seguir generando cargos.

El retiro del rol termino con DeleteRole acotado despues de confirmar la ausencia del warehouse, porque el perfil operador carece de la enumeracion ListInstanceProfilesForRole requerida por el proveedor. Se conservaron ambos intentos y el recibo IAM. La comprobacion independiente confirma ocho recursos ausentes y el plan final de Terraform no propone cambios.

Sesion autorizada: hasta una hora, objetivo USD3 sin tope de facturacion automatico. El limite Serverless y la limpieza de seguridad reducen riesgo; no reemplazan una factura. Los tiempos reales y recibos de retiro se conservan en la evidencia.

Fuentes: TRY_CAST, SVV_USER_GRANTS, Refresco, Repositorio.

Captura real: Eventos tipados y llegada

AWS Query Editor v2 · sesion del 11/11/2026 · captura manual proporcionada por el propietario. Seis validos; cinco campos JSON y llegada Kinesis actual.

Vistas ampliadas del PNG original, conservado integro en el repositorio.

SQL

```
SELECT transaction_id,event_time,arrival_timestamp,amount,payment_method,status
FROM analytics.transactions_typed
WHERE validation_error IS NULL
ORDER BY payment_method,event_time;
```

Seis eventos y llegada Kinesis

Result 1 (6)

transaction_id	event_time	arrival_timestamp	amount	payment_method	status
pe6_20261111_222257-1-0	2026-11-04 11:33:01	2026-11-11 22:23:15.561	10.1	card_pe5_20261104_113…	approved
pe6_20261111_222257-1-1	2026-11-04 11:33:02	2026-11-11 22:23:16.909	20.2	card_pe5_20261104_113…	approved
pe6_20261111_222257-1-2	2026-11-04 11:33:03	2026-11-11 22:23:18.215	999	card_pe5_20261104_113…	rejected
pe6_20261111_222257-0-0	2026-11-11 22:22:01	2026-11-11 22:23:11.827	10.1	card_pe6_20261111_222…	approved
pe6_20261111_222257-0-1	2026-11-11 22:22:02	2026-11-11 22:23:13.122	20.2	card_pe6_20261111_222…	approved
pe6_20261111_222257-0-2	2026-11-11 22:22:03	2026-11-11 22:23:14.329	999	card_pe6_20261111_222…	rejected

Captura real: Metricas por minuto

AWS Query Editor v2 · sesion del 11/11/2026 · captura manual proporcionada por el propietario. Dos grupos; tres transacciones, dos aprobadas y 30.30 aprobado por grupo.

Vistas ampliadas del PNG original, conservado integro en el repositorio.

SQL

```sql
SELECT window_start,payment_method,transaction_count,total_amount,
       average_amount,approved_count,rejected_count,approved_amount
FROM analytics.transactions_per_minute
ORDER BY window_start DESC;
```

Ventana, metodo, cantidad y total

window_start	payment_method	transaction_count	total_amount
2026-11-11 22:22:00	card_pe6_20261111_222…	3	1029.3
2026-11-04 11:33:00	card_pe5_20261104_113…	3	1029.3

Mismas dos filas: promedio, estados e importe aprobado

average_amount	approved_count	rejected_count	approved_amount
343.1	2	1	30.3
343.1	2	1	30.3

Captura real: Conciliacion con Iceberg

AWS Query Editor v2 · sesion del 11/11/2026 · captura manual proporcionada por el propietario. Dos aprobadas y 30.30 por lado; diferencias cero.

Vistas ampliadas del PNG original, conservado integro en el repositorio.

SQL

```sql
SELECT payment_method,recent_approved_count,iceberg_approved_count,
       recent_approved_amount,iceberg_approved_amount,
       count_difference,amount_difference
FROM analytics.recent_vs_iceberg;
```

Metodo y cantidades aprobadas

payment_method	recent_approved_co…	iceberg_approved_c…
card_pe5_20261104_113…	2	2

Misma fila: importes y diferencias

recent_approved_am…	iceberg_approved_a…	count_difference	amount_difference
30.3	30.3	0	0

Captura real: Membresia efectiva

AWS Query Editor v2 · sesion del 11/11/2026 · captura manual proporcionada por el propietario. La identidad de consola tiene checkpoint6_analyst sin admin_option.

Vistas ampliadas del PNG original, conservado integro en el repositorio.

SQL

```sql
SELECT current_user AS effective_user,user_name,role_name,admin_option
FROM svv_user_grants
WHERE user_name=current_user AND role_name='checkpoint6_analyst';
```

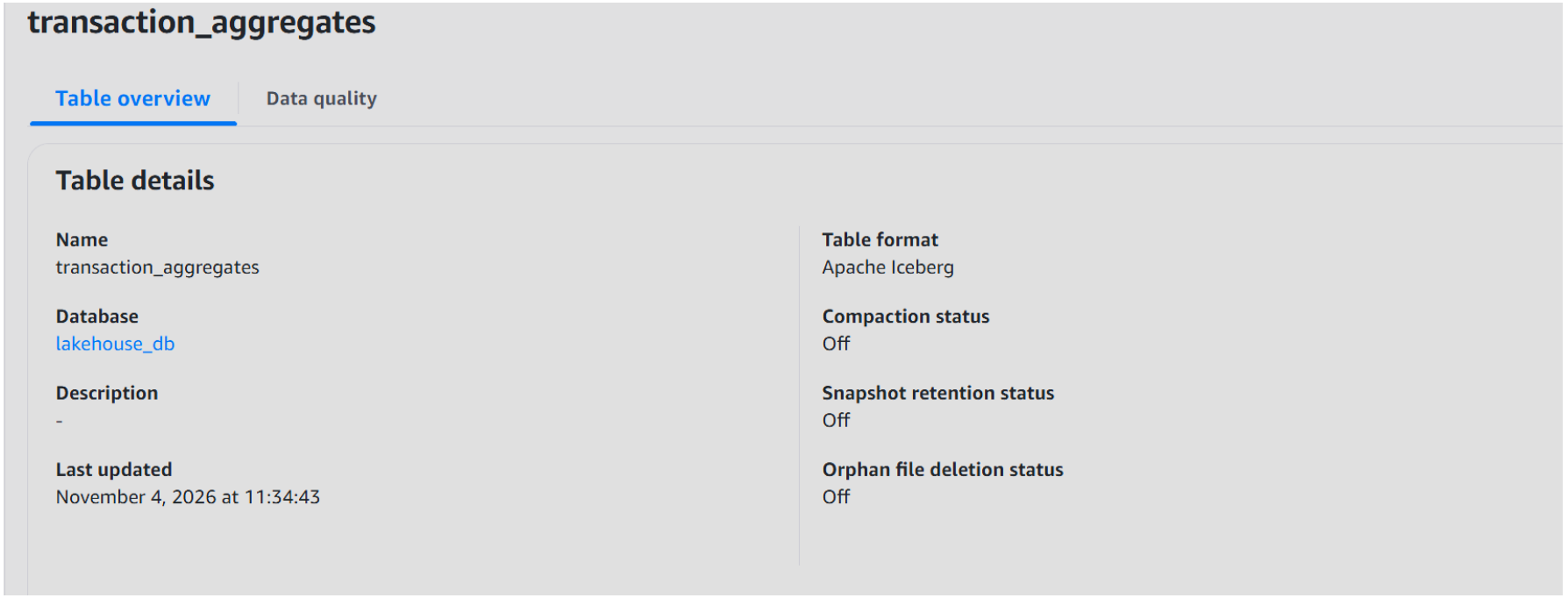
Usuario efectivo, rol y admin_option

Result 1 (1)

effective_user	user_name	role_name	admin_option
IAM:RootIdentity	IAM:RootIdentity	checkpoint6_analyst	false

Metadatos Iceberg: antecedente PE5

AWS Glue Data Catalog. PNG original de PE5; fecha de captura no embebida. Muestra formato Iceberg, base y seis columnas. El ensayo nuevo leyo esa tabla mediante el esquema externo.



Anexo: SQL fuente de esta revisión

Archivo ejecutable: sql/redshift/checkpoint_6_complete.sql. SHA256: b51176a029026b80b8bbac5f6147fbe56ab724ec7b1df64e811b174c0ec8e30c

01_schemas.sql

```
-- Execute as deployment owner, database analytics, region us-east-1.
-- No CREATE EXTERNAL DATABASE: Glue already exists and remains Terraform-owned.
SET timezone TO 'UTC';
CREATE SCHEMA IF NOT EXISTS streaming;
CREATE SCHEMA IF NOT EXISTS analytics;
CREATE EXTERNAL SCHEMA IF NOT EXISTS kinesis_source
FROM KINESIS IAM_ROLE default REGION 'us-east-1';
CREATE EXTERNAL SCHEMA IF NOT EXISTS lakehouse
FROM DATA CATALOG DATABASE 'lakehouse_db'
IAM_ROLE default REGION 'us-east-1';
```

02_streaming.sql

```
-- Exactly one base MV consumes the existing stream. Refresh is explicitly manual.
-- Keep invalid JSON bytes for diagnostics; CASE guards JSON_PARSE.
-- Business conversions are downstream, so a bad amount cannot block ingestion.
CREATE MATERIALIZED VIEW streaming.transactions_raw
AUTO REFRESH NO AS
SELECT approximate_arrival_timestamp, refresh_time, partition_key,
       shard_id, sequence_number, kinesis_data,
       CAN_JSON_PARSE(kinesis_data) AS valid_json,
       CASE WHEN CAN_JSON_PARSE(kinesis_data)
            THEN JSON_PARSE(kinesis_data) ELSE NULL END AS payload
FROM kinesis_source."dev-transactions-stream";
```

03_analytics.sql

```
-- SUPER paths correspond to real transaction JSON, not Java POJO field names.
-- Empty strings are normalized before TRY_CAST. Objects/arrays become NULL.
CREATE OR REPLACE VIEW analytics.transactions_typed AS
WITH fields AS (
  SELECT approximate_arrival_timestamp, refresh_time, shard_id, sequence_number,
         valid_json,
         NULLIF(TRIM(payload.transaction_id::varchar), '') AS transaction_id,
         NULLIF(TRIM(payload.payment_method::varchar), '') AS payment_method,
         LOWER(NULLIF(TRIM(payload.status::varchar), '')) AS status,
         NULLIF(TRIM(payload.amount::varchar), '') AS amount_text,
         NULLIF(TRIM(payload."timestamp"::varchar), '') AS timestamp_text
  FROM streaming.transactions_raw
), converted AS (
  SELECT *, TRY_CAST(amount_text AS decimal(20,4)) AS precise_amount,
         CASE WHEN timestamp_text ~ '^[0-9]{13}$'
              THEN DATEADD(millisecond, MOD(TRY_CAST(timestamp_text AS bigint),86400000)::integer,
                   DATEADD(day, (TRY_CAST(timestamp_text AS bigint)/86400000)::integer,
                           TIMESTAMP '1970-01-01 00:00:00'))
              -- ISO 8601 requires an explicit timezone; TRY_CAST checks the date.
              WHEN timestamp_text ~ (
                   '^[0-9]{4}-[0-9]{2}-[0-9]{2}[T ][0-9]{2}:[0-9]{2}:[0-9]{2}'
                   || '([.][0-9]{1,6})?(Z|[+-][0-9]{2}:[0-9]{2})$')
              THEN TRY_CAST(timestamp_text AS timestamptz) AT TIME ZONE 'UTC'
              ELSE NULL
         END AS event_time
  FROM fields
)
SELECT approximate_arrival_timestamp, refresh_time, shard_id, sequence_number,
       transaction_id, payment_method, status, event_time,
       TRY_CAST(precise_amount AS decimal(18,2)) AS amount,
       CASE WHEN NOT valid_json THEN 'invalid_json'
            WHEN transaction_id IS NULL OR payment_method IS NULL OR status IS NULL
              THEN 'missing_required_field'
            WHEN amount_text IS NULL
              OR NOT (amount_text ~ '^[0-9]+([.][0-9]{1,2})?$')
              OR precise_amount IS NULL OR precise_amount < 0
              OR precise_amount <> ROUND(precise_amount,2)
```

```
              OR TRY_CAST(precise_amount AS decimal(18,2)) IS NULL THEN 'invalid_amount'
            WHEN event_time IS NULL THEN 'invalid_timestamp'
            ELSE NULL END AS validation_error,
       approximate_arrival_timestamp AS arrival_timestamp
FROM converted;

-- Ordinary aggregate view avoids assumptions about incremental eligibility of
-- SUPER extraction/timestamp conversions in a nested MV. Always reads base state.
CREATE OR REPLACE VIEW analytics.transactions_per_minute AS
SELECT DATE_TRUNC('minute',event_time) AS window_start, payment_method,
       COUNT(*) AS transaction_count,
       SUM(amount) AS total_amount,
       -- AVG preserves input scale: widen BEFORE aggregating, not afterwards.
       AVG(amount::decimal(24,6))::decimal(38,6) AS average_amount,
       SUM(CASE WHEN status='approved' THEN 1 ELSE 0 END) AS approved_count,
       SUM(CASE WHEN status='rejected' THEN 1 ELSE 0 END) AS rejected_count,
       SUM(CASE WHEN status='approved' THEN amount ELSE 0 END) AS approved_amount
FROM analytics.transactions_typed
WHERE validation_error IS NULL
GROUP BY 1,2;
```

04_iceberg.sql

```
-- Iceberg contains approved aggregates, NOT individual transactions.
-- Both sides are collapsed to exactly one row per UTC minute/payment method.
-- Historical duplicates are summed, not hidden: drift should expose replay.
-- Late data and Flink watermark/checkpoint timing can temporarily cause drift.
CREATE OR REPLACE VIEW analytics.recent_vs_iceberg AS
WITH recent AS (
  SELECT DATE_TRUNC('minute',event_time) AS window_start, payment_method,
         COUNT(*) AS approved_count, SUM(amount) AS approved_amount
  FROM analytics.transactions_typed
  WHERE validation_error IS NULL AND status='approved'
  GROUP BY 1,2
), historical AS (
  SELECT window_start, payment_method,
         SUM(transaction_count) AS approved_count, SUM(total_amount) AS approved_amount
  FROM lakehouse.transaction_aggregates
  GROUP BY 1,2
)
SELECT r.window_start, r.payment_method,
       r.approved_count AS recent_approved_count,
       h.approved_count AS iceberg_approved_count,
       r.approved_amount AS recent_approved_amount,
       h.approved_amount AS iceberg_approved_amount,
       r.approved_count-h.approved_count AS count_difference,
       r.approved_amount-h.approved_amount AS amount_difference
FROM recent r JOIN historical h
ON r.window_start=h.window_start AND r.payment_method=h.payment_method
WITH NO SCHEMA BINDING;
```

05_security.sql

```
-- Run once as owner. AWS IAM role grants service access; this DB role grants SQL.
CREATE ROLE checkpoint6_analyst;
REVOKE ALL ON SCHEMA streaming FROM PUBLIC;
REVOKE ALL ON SCHEMA analytics FROM PUBLIC;
REVOKE ALL ON SCHEMA kinesis_source FROM PUBLIC;
REVOKE ALL ON SCHEMA lakehouse FROM PUBLIC;
GRANT USAGE ON SCHEMA analytics TO ROLE checkpoint6_analyst;
-- Late-binding view resolution also needs USAGE on its external schema.
-- The IAM execution role limits external reads to the single Iceberg table.
GRANT USAGE ON SCHEMA lakehouse TO ROLE checkpoint6_analyst;
GRANT SELECT ON analytics.transactions_typed TO ROLE checkpoint6_analyst;
GRANT SELECT ON analytics.transactions_per_minute TO ROLE checkpoint6_analyst;
GRANT SELECT ON analytics.recent_vs_iceberg TO ROLE checkpoint6_analyst;
-- Analyst cannot refresh MVs, create schemas, read raw bytes, or assume IAM role.
-- Explicitly bind a real authenticated DB identity after approving membership:
-- GRANT ROLE checkpoint6_analyst TO "IAM:analyst-name";
```

06_validation.sql

```
-- Manual sequence: publish test records, refresh once, query; no one-second loop.
REFRESH MATERIALIZED VIEW streaming.transactions_raw;
SELECT transaction_id,event_time,amount,payment_method,status,validation_error
FROM analytics.transactions_typed ORDER BY approximate_arrival_timestamp DESC LIMIT 20;
SELECT * FROM analytics.transactions_per_minute ORDER BY window_start DESC LIMIT 20;
SELECT * FROM lakehouse.transaction_aggregates
WHERE window_start >= TIMESTAMP '2026-11-04 00:00:00'
  AND window_start < TIMESTAMP '2026-11-05 00:00:00';
-- A comparison, not a UNION of overlapping histories; do not add the two totals.
SELECT * FROM analytics.recent_vs_iceberg ORDER BY window_start DESC LIMIT 20;
SELECT * FROM svv_role_grants WHERE role_name='checkpoint6_analyst';
-- User membership is demonstrated after the explicit binding in SQL08.
SELECT * FROM svv_relation_privileges WHERE identity_name='checkpoint6_analyst';
```

07_monitoring.sql

```
-- Serverless-compatible SYS views. Owner sees their own refreshes; superuser all.
SELECT * FROM sys_stream_scan_states
WHERE external_schema_name='kinesis_source' ORDER BY record_time DESC LIMIT 20;
SELECT * FROM sys_stream_scan_errors
WHERE external_schema_name='kinesis_source' ORDER BY record_time DESC LIMIT 20;
SELECT mv_name,refresh_type,status,start_time,end_time,duration
FROM sys_mv_refresh_history
WHERE schema_name='streaming' ORDER BY start_time DESC LIMIT 20;
SELECT * FROM sys_mv_state ORDER BY start_time DESC LIMIT 20;
SELECT COUNT(*) AS stored_records, MAX(approximate_arrival_timestamp) AS last_arrival,
       MAX(refresh_time) AS last_ingest_refresh
FROM streaming.transactions_raw;
SELECT validation_error,COUNT(*) AS records
FROM analytics.transactions_typed GROUP BY 1;
-- refresh_time is refresh start, not record visibility time. Negative values are
-- possible for records arriving during a refresh. Data API observation separately
-- measures upper-bound end-to-end latency; never present this as an SLA.
SELECT shard_id,sequence_number,approximate_arrival_timestamp,refresh_time,
       DATEDIFF(millisecond,approximate_arrival_timestamp,GETDATE()) AS arrival_age_ms
FROM streaming.transactions_raw ORDER BY approximate_arrival_timestamp DESC LIMIT 20;
```

08_membership.sql

```
-- Run ONCE in the clean temporary warehouse as its deployment owner.
-- Password-disabled SQL test identity: no AWS/IAM user or password is created.
CREATE USER checkpoint6_access_test PASSWORD DISABLE;
GRANT ROLE checkpoint6_analyst TO checkpoint6_access_test;

-- Explicit membership, separate from role-to-role and object privilege grants.
SELECT user_name,role_name,admin_option
FROM svv_user_grants
WHERE user_name='checkpoint6_access_test'
  AND role_name='checkpoint6_analyst';

-- Keep these statements in ONE session. Owner must be allowed to switch identity.
SET SESSION AUTHORIZATION checkpoint6_access_test;
SELECT current_user AS effective_user;
SELECT user_name,role_name,admin_option
FROM svv_user_grants
WHERE user_name=current_user AND role_name='checkpoint6_analyst';
SELECT COUNT(*) AS visible_metric_rows FROM analytics.transactions_per_minute;
SELECT COUNT(*) AS visible_join_rows FROM analytics.recent_vs_iceberg;
RESET SESSION AUTHORIZATION;

-- Expected-denial tests are in redshift-security-smoke.py, not unhandled errors
-- in this consolidated script. Capture membership + effective_user + allow/deny.
```